

6.4 Training Neural Networks with multiple inputs

This notebook is adapted from the code taken from

https://github.com/Ceyron/machine-learning-and-simulation/blob/main/english/neural_network_sine_learning/nn_learns_sine_function_wit

In Notebook 3.6, there was an Class Demo that asked to use $f_{model}(x_1, x_2) = a_0 + a_1x_1 + a_2x_2 + a_3x_1x_2 + a_4x_1^2 + a_5x_2^2$ to fit the function $y = \sin(x_1x_2)$. We will do the same problem again here. You will find that you could not get a good fit mainly because $f_{model}(x_1, x_2)$ is not a good general function approximator. Later in this notebook, we will try to use a relatively complex Neural Network to see if we fit it better to the data generated from $y = \sin(x_1x_2)$.

This notebook uses the Neural Network package in Julia, `Lux.jl` to represent the Neural network.

Importing all packages

```
In [5]: using Lux
using Random
using Zygote
using Optimisers
using CairoMakie
using Distributions
using Statistics
using Printf
using Optimization
using OptimizationOptimisers
using ComponentArrays
```

Draw samples to create a toy data set $y = \sin(x_1x_2)$. Here, we will generate our own data.

```
In [6]: N_SAMPLES = 1_000

# Our Pseudo-Random Number Generator
rng = Xoshiro(42)

x1_samples = rand(
    rng,
    Uniform(0.0, 2 * π),
    (1, N_SAMPLES),
)

x2_samples = rand(
    rng,
    Uniform(0.0, 2 * π),
```

```

    (1,N_SAMPLES),
)

y_noise = rand(
    rng,
    Normal(0.0, 0.01),
    (1,N_SAMPLES),
)

y_samples = Float32.(sin.(x1_samples.*x2_samples) .+ 0.01*y_noise)
x_samples=[Float32.(x1_samples);Float32.(x2_samples)]

```

2×1000 Matrix{Float32}:

```

1.0906  2.02106  1.62474  1.04576  ...  4.90091  4.47217  5.38323  1.0641
2.65313  4.63591  0.633562  2.01732  ...  5.54129  3.88784  5.62645  0.7667
78

```

The problem that we are interested in has two inputs and 1 output. PLEASE note that the input `x_samples` is a vector of matrix size $2 \times N_SAMPLES$ and the output `y_samples` is a vector of matrix of size $1 \times N_SAMPLES$. If you use the `Lux.jl` package, it expects data to be ordered like that

$$\mathbf{x} = \begin{bmatrix} x_{11} & x_{12} & x_{13} & \cdots & x_{1N} \\ x_{21} & x_{22} & x_{23} & \cdots & x_{2N} \end{bmatrix}$$

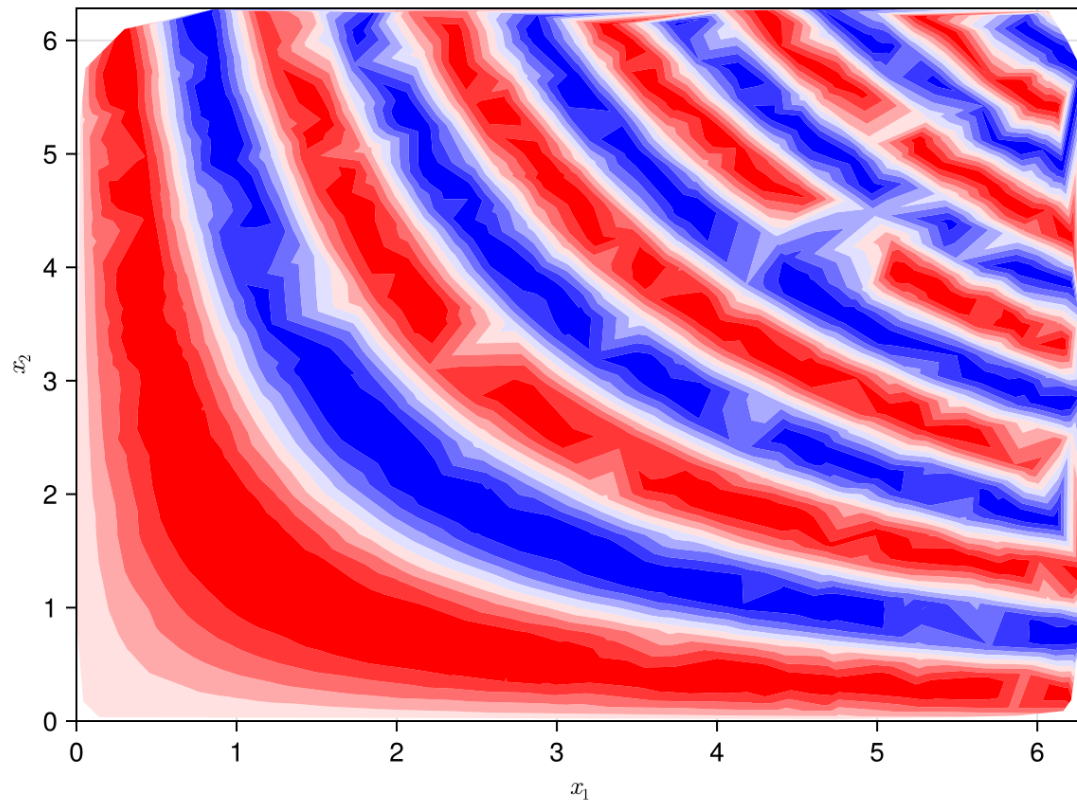
$$\mathbf{y} = [y_1 \quad y_2 \quad y_3 \quad \cdots \quad y_N]$$

Plot the data to see how it looks like

```

In [7]: fig = Figure()
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"x_1", ylabel = L"x_2", limits=(
    tricontourf!(ax,x_samples[1,:],x_samples[2,:],y_samples[:],colormap=:bwr)
fig

```



Let's now see if we can use a polynomial

$$f_{model}(x_1, x_2) = a_0 + a_1x_1 + a_2x_2 + a_3x_1x_2 + a_4x_1^2 + a_5x_2^2$$

to fit this function. We will use the normal equations to find the coefficients. First construct the $[V]$ matrix

$$\mathbf{V} = \begin{bmatrix} 1 & x_{11} & x_{21} & x_{11}x_{21} & x_{11}^2 & x_{21}^2 \\ 1 & x_{12} & x_{22} & x_{12}x_{22} & x_{12}^2 & x_{22}^2 \\ 1 & x_{13} & x_{23} & x_{13}x_{23} & x_{13}^2 & x_{23}^2 \\ \vdots & & & & & \\ 1 & x_{1N} & x_{2N} & x_{1N}x_{2N} & x_{1N}^2 & x_{2N}^2 \end{bmatrix}$$

Then we can find the coefficients using the formula

$$\vec{a} = (\mathbf{V}^T \mathbf{V})^{-1} \mathbf{V}^T \vec{y}$$

where $\vec{y}$ is the quantity that you are trying to predict.

```
In [8]: V=hcant(ones(N_SAMPLES),x1_samples[:,],x2_samples[:,],x1_samples[:,].*x2_samp
a=V'*V\ (V'*y_samples[:,])
```

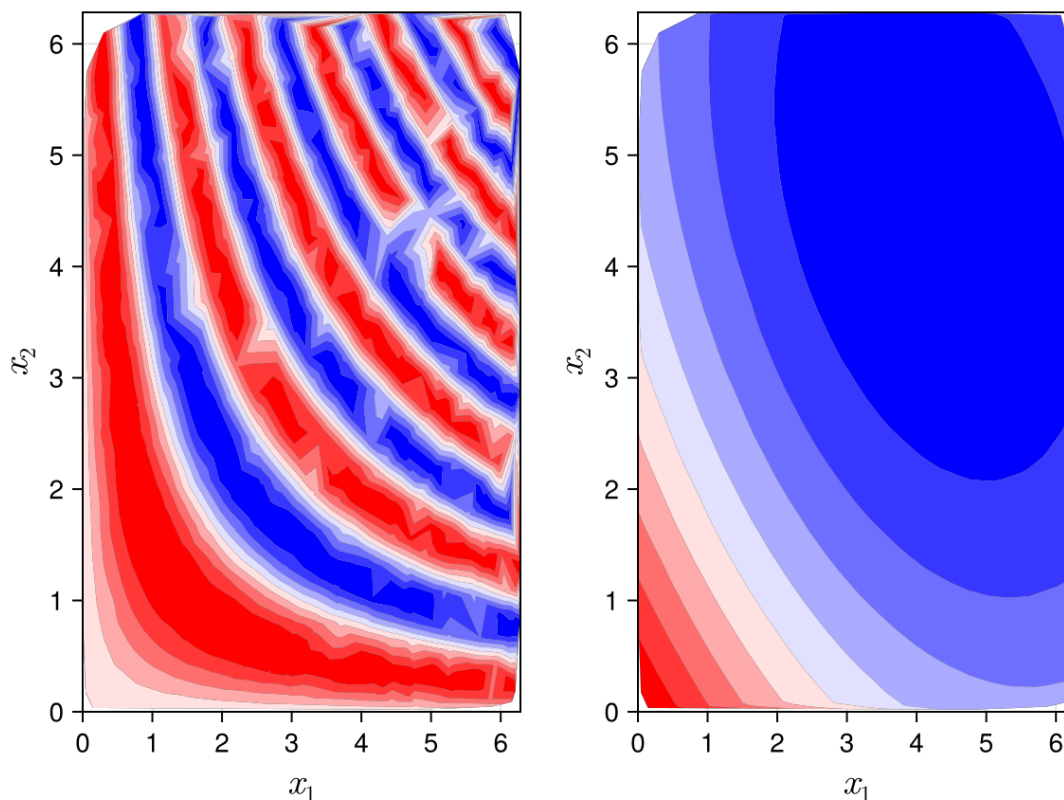
```
6-element Vector{Float64}:
 0.7173151855030183
-0.19110918250673548
-0.16981952199684316
 0.010613206345325818
 0.01685952549659496
 0.01359615749223827
```

Then compute the prediction from f_{model}

```
In [9]: ypred=a[1].+a[2]*x1_samples[:,].+a[3]*x2_samples[:,].+a[4]*x1_samples[:,].*x
```

```
1000-element Vector{Float64}:
 0.20480515696096138
 0.004313921550534505
 0.36010905306287266
 0.2710387763412488
-0.046515792594417515
 0.2815401206896467
-0.026294216238426593
-0.006513504096515951
-0.017021925991159814
 0.13267018321217608
 ⋮
 0.18562579110990549
 0.5993123725545937
 0.05540093227250331
 0.20812941183664696
-0.029361296308365703
-0.049656608112769896
-0.0703504461495568
-0.026504621680687512
 0.41948527942760183
```

```
In [10]: fig = Figure(backgroundcolor=:transparent)
ax1 = CairoMakie.Axis(fig[1, 1], xlabel = L"x_1", ylabel = L"x_2", limits=
ax2 = CairoMakie.Axis(fig[1, 2], xlabel = L"x_1", ylabel = L"x_2", limits=
tricontourf!(ax1,x_samples[1,:],x_samples[2,:],y_samples[:,],colormap=:bwr)
tricontourf!(ax2,x_samples[1,:],x_samples[2,:],ypred,colormap=:bwr)
save("../figures/06p04TrainingMultivariateNeuralNetworks01.png",fig)
display(fig)
```



CairoMakie.Screen{IMAGE}

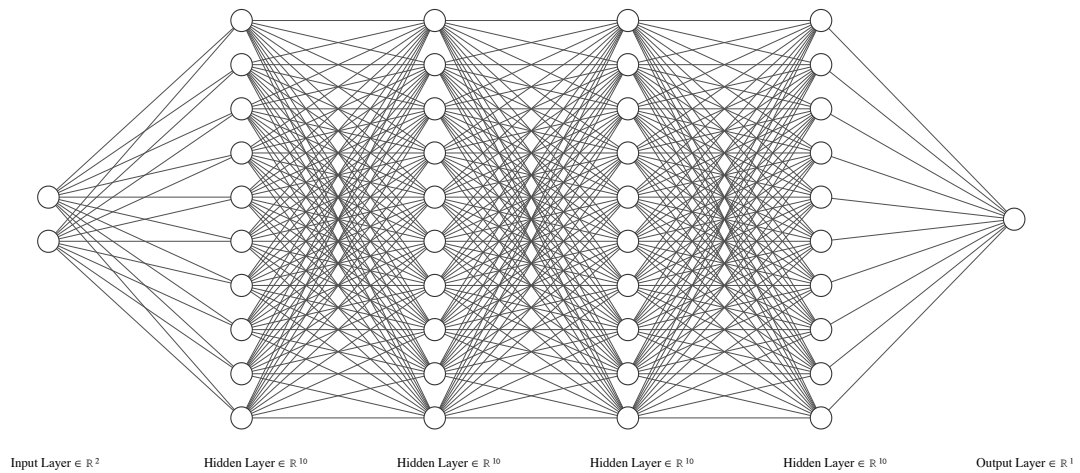
As you can see, the prediction using f_{model} is not very good. Now let's try to get a better prediction using a Neural Network.

The code below defines the Neural Network Architecture. This Neural Network has two inputs and one output with 3 hidden layers with ten nodes. This Neural Network is quite complex and should be able model $y = \sin(x_1 x_2)$ much better than the 2nd order polynomial $f_{model}()$ function in notebook 3.6.

```
In [11]: model = Chain(
    Dense(2 => 10),
    Dense(10 => 10, tanh),
    Dense(10 => 10, tanh),
    Dense(10 => 10, tanh),
    Dense(10 => 1)
)

Chain(
  layer_1 = Dense(2 => 10),           # 30 parameters
  layer_(2-4) = Dense(10 => 10, tanh), # 330 (110 x 3) paramete
rs
  layer_5 = Dense(10 => 1),           # 11 parameters
)                                     # Total: 371 parameters,
                                     # plus 0 states.
```

A visual representation of the Neural Network is shown below.



(this figure was made using <https://alexlenail.me/NN-SVG/>)

Information about the network is printed above. It has 4 hidden layers, with 371 unknown parameters which we are able to tune with some optimization technique.

The code below initializes the parameters of the network with some random numbers

```
In [12]: parameters, layer_states = Lux.setup(rng, model)

((layer_1 = (weight = Float32[-1.0708317 -1.1302087; -0.6040637 -1.195622
8; ... ; -0.2955299 -0.4776289; -0.7644195 0.7865141], bias = Float32[-0.632
1535, -0.6105538, 0.5708269, 0.0036835503, 0.21545537, 0.016626425, 0.1774
089, -0.21669406, 0.4291924, 0.7043529]), layer_2 = (weight = Float32[-0.7
067458 0.59171075 ... -0.19577038 0.29535174; -0.35265473 0.21456157 ... 0.744
24547 -0.37957725; ... ; 0.5083448 -0.056543726 ... -0.89865136 0.1288232; 0.7
259346 -0.85188794 ... 0.2754867 0.45333225], bias = Float32[0.015074015, -
0.2217923, 0.09363705, -0.17262654, -0.29449806, 0.059713297, 0.25072336,
0.012980231, -0.11212885, -0.22559524]), layer_3 = (weight = Float32[0.690
135 0.011821191 ... -0.28916267 0.2724587; -0.11111863 -0.8536415 ... -0.60466
26 0.28117096; ... ; -0.28098673 -0.5443014 ... 0.6362686 0.560396; 0.3686039
0.45359918 ... 0.23107682 0.746915], bias = Float32[-0.06421895, 0.10363632
4, -0.17465466, -0.009129529, 0.15563653, 0.2127711, -0.29570895, -0.18826
15, -0.16013762, -0.044853814]), layer_4 = (weight = Float32[0.80452514 -
0.18739125 ... -0.13405693 0.012745748; -0.10613216 0.35289282 ... 0.33380327
0.287197; ... ; -0.10101161 -0.38968632 ... -0.4180415 0.014271008; -0.5345931
-0.41970757 ... 0.03462281 0.36636746], bias = Float32[-0.22321966, 0.299461
63, 0.097026184, 0.2526037, -0.087348685, 0.07064649, -0.06024939, 0.04119
4655, 0.31104526, 0.28265205]), layer_5 = (weight = Float32[-0.09843542 -
0.4028636 ... -0.055847723 -0.42583376], bias = Float32[0.0311288])), (layer
_1 = NamedTuple(), layer_2 = NamedTuple(), layer_3 = NamedTuple(), layer_4
= NamedTuple(), layer_5 = NamedTuple()))
```

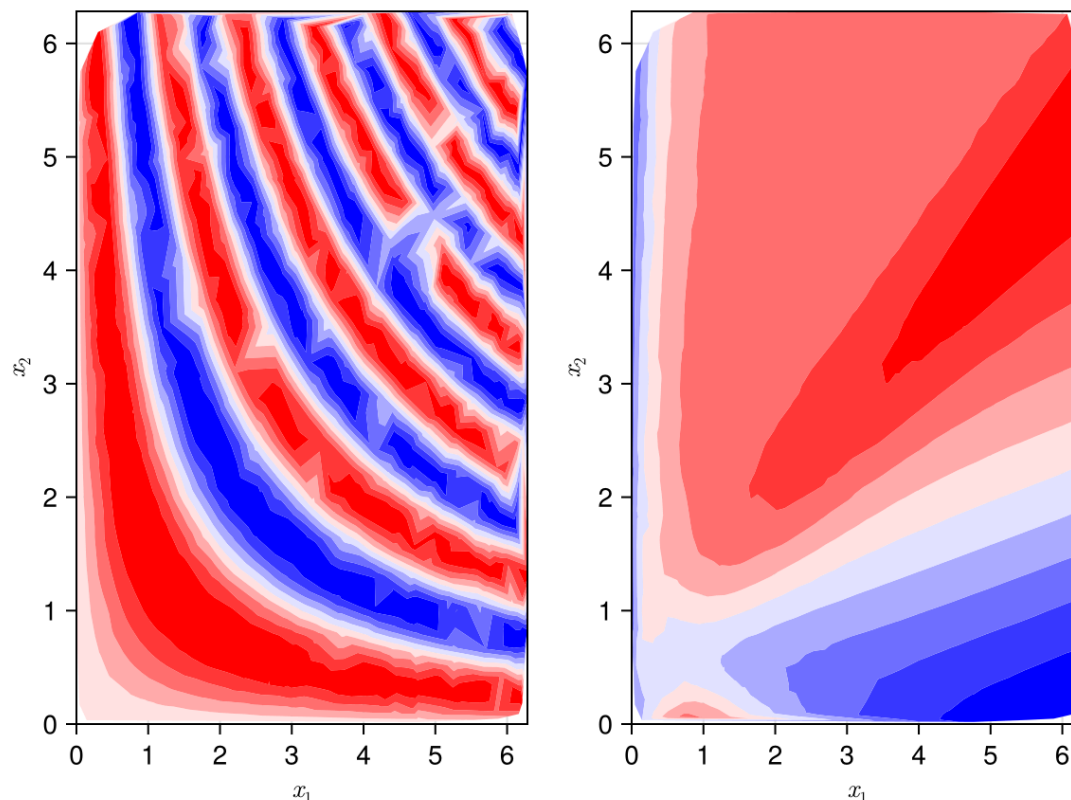
Now plot the data and the initial (guess) prediction just to see how far off you are from the actual data.

```
In [13]: fig = Figure()
ax1 = CairoMakie.Axis(fig[1, 1], xlabel = L"x_1", ylabel = L"x_2", limits=
tricontourf!(ax1,x_samples[1,:],x_samples[2,:],y_samples[:,],colormap=:bwr

# predict using Neural Network
y_pred,_=model(x_samples,parameters, layer_states) #note that model expect
```

```
ax2 = CairoMakie.Axis(fig[1, 2], xlabel = L"x_1", ylabel = L"x_2", limits=
tricontourf!(ax2,x_samples[1,:],x_samples[2:],y_pred[:],colormap=:bwr)
```

```
fig
```



The left figure is the data and the right figure is the initial neural network prediction. The prediction is not good which is expected because the parameters are just random.

Now we will try to find parameters of the Neural Network such that it best fits the data. First, define the loss function

```
In [14]: function loss_fn(p, ls)
           y_prediction, new_ls= model(x_samples, p, ls)
           loss = 0.5 * mean((y_prediction .- y_samples).^2)
           return loss
       end
```

loss_fn (generic function with 1 method)

The code below trains the Neural Network using the standard optimisation functions in Julia

```
In [15]: callback = function (p, l)
           if length(loss_history) % 100 == 0
               println("Iteration: $(p.iter), Loss: $l")
           end
           push!(loss_history, l)
           return false
       end
```

```
end

adtype = Optimization.AutoZygote()
optf = Optimization.OptimizationFunction(loss_fn, adtype)
optprob = Optimization.OptimizationProblem(optf, ComponentArray(parameter
#optprob = Optimization.OptimizationProblem(optf, ComponentArray(neural_n

# Initialize a vector to store loss values
loss_history = Float64[]
neural_network = Optimization.solve(optprob, OptimizationOptimisers.Adam(
```


Iteration: 1, Loss: 0.24302177131175995
Iteration: 101, Loss: 0.16576603055000305
Iteration: 201, Loss: 0.140061154961586
Iteration: 301, Loss: 0.11703614145517349
Iteration: 401, Loss: 0.11082131415605545
Iteration: 501, Loss: 0.0969199612736702
Iteration: 601, Loss: 0.09483771026134491
Iteration: 701, Loss: 0.08447102457284927
Iteration: 801, Loss: 0.08343677222728729
Iteration: 901, Loss: 0.0782032459974289
Iteration: 1001, Loss: 0.07582271844148636
Iteration: 1101, Loss: 0.07442113757133484
Iteration: 1201, Loss: 0.07308414578437805
Iteration: 1301, Loss: 0.07352937757968903
Iteration: 1401, Loss: 0.0737283006310463
Iteration: 1501, Loss: 0.061261724680662155
Iteration: 1601, Loss: 0.06119990721344948
Iteration: 1701, Loss: 0.05919545516371727
Iteration: 1801, Loss: 0.05321675166487694
Iteration: 1901, Loss: 0.052232030779123306
Iteration: 2001, Loss: 0.045066025108098984
Iteration: 2101, Loss: 0.04693209379911423
Iteration: 2201, Loss: 0.03999995067715645
Iteration: 2301, Loss: 0.03768927976489067
Iteration: 2401, Loss: 0.037988483905792236
Iteration: 2501, Loss: 0.039974506944417953
Iteration: 2601, Loss: 0.03832200914621353
Iteration: 2701, Loss: 0.03583309426903725
Iteration: 2801, Loss: 0.038940176367759705
Iteration: 2901, Loss: 0.03774011880159378
Iteration: 3001, Loss: 0.03929688408970833
Iteration: 3101, Loss: 0.03722987696528435
Iteration: 3201, Loss: 0.03524910286068916
Iteration: 3301, Loss: 0.03712578862905502
Iteration: 3401, Loss: 0.038499101996421814
Iteration: 3501, Loss: 0.035487670451402664
Iteration: 3601, Loss: 0.03493860736489296
Iteration: 3701, Loss: 0.036605097353458405
Iteration: 3801, Loss: 0.03499896824359894
Iteration: 3901, Loss: 0.03844665735960007
Iteration: 4001, Loss: 0.03540882468223572
Iteration: 4101, Loss: 0.03978641331195831
Iteration: 4201, Loss: 0.03591420128941536
Iteration: 4301, Loss: 0.03720906376838684
Iteration: 4401, Loss: 0.034607887268066406
Iteration: 4501, Loss: 0.03758619353175163
Iteration: 4601, Loss: 0.03475935012102127
Iteration: 4701, Loss: 0.04603162035346031
Iteration: 4801, Loss: 0.03476673364639282
Iteration: 4901, Loss: 0.034495603293180466
Iteration: 5001, Loss: 0.03607343137264252
Iteration: 5101, Loss: 0.039128709584474564
Iteration: 5201, Loss: 0.03954470902681351
Iteration: 5301, Loss: 0.03758306801319122
Iteration: 5401, Loss: 0.03358418121933937
Iteration: 5501, Loss: 0.035126328468322754
Iteration: 5601, Loss: 0.03180506452918053
Iteration: 5701, Loss: 0.03036101721227169
Iteration: 5801, Loss: 0.045454178005456924
Iteration: 5901, Loss: 0.030209219083189964

Iteration: 6001, Loss: 0.030366258695721626
Iteration: 6101, Loss: 0.03143380582332611
Iteration: 6201, Loss: 0.032507628202438354
Iteration: 6301, Loss: 0.03508559614419937
Iteration: 6401, Loss: 0.03439272195100784
Iteration: 6501, Loss: 0.029863931238651276
Iteration: 6601, Loss: 0.031174104660749435
Iteration: 6701, Loss: 0.030883297324180603
Iteration: 6801, Loss: 0.02983514405786991
Iteration: 6901, Loss: 0.0392850786447525
Iteration: 7001, Loss: 0.030597232282161713
Iteration: 7101, Loss: 0.03153374046087265
Iteration: 7201, Loss: 0.03367488831281662
Iteration: 7301, Loss: 0.030422668904066086
Iteration: 7401, Loss: 0.02950485423207283
Iteration: 7501, Loss: 0.02944774180650711
Iteration: 7601, Loss: 0.029200637713074684
Iteration: 7701, Loss: 0.02850300818681717
Iteration: 7801, Loss: 0.03007487766444683
Iteration: 7901, Loss: 0.030627800151705742
Iteration: 8001, Loss: 0.02878398820757866
Iteration: 8101, Loss: 0.02937808260321617
Iteration: 8201, Loss: 0.02872651442885399
Iteration: 8301, Loss: 0.0351872555911541
Iteration: 8401, Loss: 0.031637854874134064
Iteration: 8501, Loss: 0.02757440321147442
Iteration: 8601, Loss: 0.030115492641925812
Iteration: 8701, Loss: 0.027909589931368828
Iteration: 8801, Loss: 0.02748473361134529
Iteration: 8901, Loss: 0.028822237625718117
Iteration: 9001, Loss: 0.02820543944835663
Iteration: 9101, Loss: 0.027550775557756424
Iteration: 9201, Loss: 0.03166421875357628
Iteration: 9301, Loss: 0.026963509619235992
Iteration: 9401, Loss: 0.026761583983898163
Iteration: 9501, Loss: 0.026671428233385086
Iteration: 9601, Loss: 0.026885392144322395
Iteration: 9701, Loss: 0.026423804461956024
Iteration: 9801, Loss: 0.02770293690264225
Iteration: 9901, Loss: 0.028347937390208244
Iteration: 10000, Loss: 0.026174133643507957

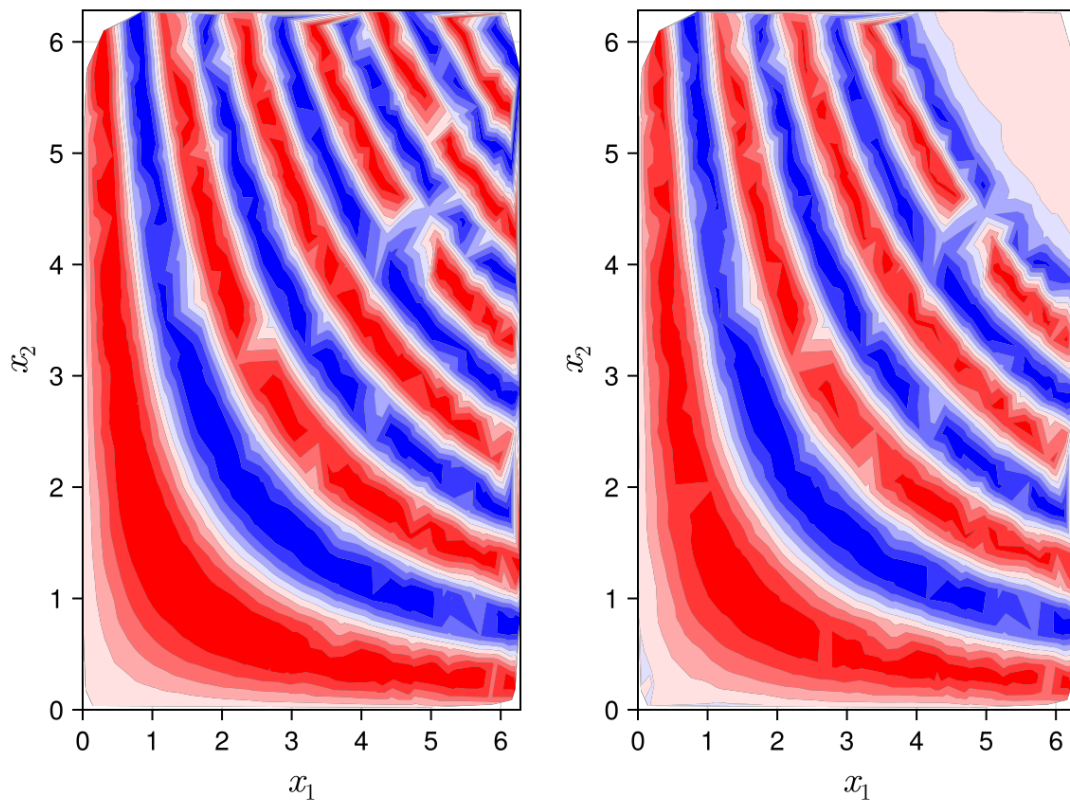
retcode: Default

```
u: ComponentVector{Float32}(layer_1 = (weight = Float32[-0.8846789 -0.7586
244; -0.7583663 -1.0005567; ... ; -0.35511908 -0.68568027; -0.64021146 0.571
5531], bias = Float32[-0.80012256, 0.39184767, 0.30313408, 0.2547064, 0.94
97092, -1.3695402, 0.5040136, -0.92151296, 1.533342, 0.21242887]), layer_2
= (weight = Float32[-0.89147335 0.3842042 ... -0.7776079 0.6171096; -0.40263
313 0.17958526 ... 1.176593 0.23257558; ... ; 0.20941211 -0.4245439 ... -1.41766
97 0.13997711; 0.87328905 -0.76277125 ... 0.45063 0.47768763], bias = Float3
2[0.3494335, 0.71557915, 0.8832311, 0.070235506, 1.0107183, -0.40142313, -
0.124759816, -0.33796743, 0.08076274, -0.28548646]), layer_3 = (weight = F
loat32[0.5292928 0.5053655 ... -0.8239256 0.53027993; -0.63929135 -3.8994963
... -0.24583776 0.17200913; ... ; -0.33763108 -0.90386325 ... 1.0590286 0.499500
04; 0.4015763 0.101341836 ... 0.4609156 0.7822772], bias = Float32[-0.139839
52, -1.0606837, -0.14822577, 0.28656882, -0.07451504, 0.54519624, -0.82665
83, -0.28146675, -0.6270402, 0.04047111]), layer_4 = (weight = Float32[0.4
8155478 -0.3657515 ... -0.036487844 -1.076374; -0.67695296 -0.23321232 ... 1.0
626457 0.8367781; ... ; -0.8443649 3.6847594 ... 0.28482592 1.4729549; -0.5487
937 -1.2695051 ... -0.23237635 0.84635013], bias = Float32[-0.65195584, 0.67
31633, -0.36068618, -0.91588944, -0.029304681, -1.2588303, -0.02603273, 0.
49755743, 0.84855145, -0.2326538]), layer_5 = (weight = Float32[-1.9233638
-1.5270581 ... 1.0892116 -2.9700587], bias = Float32[0.014239239]))
```

Now we plot the original data and the prediction from the Neural Network to see if the prediction is a better match with the data (compared with the prediction using the initial random parameters)

```
In [16]: # predict using Neural Network
y_NN, _ = model(x_samples, neural_network.u, layer_states) #note that model e

fig = Figure(backgroundcolor=:transparent)
ax1 = CairoMakie.Axis(fig[1, 1], xlabel = L"x_1", ylabel = L"x_2", limits=
ax2 = CairoMakie.Axis(fig[1, 2], xlabel = L"x_1", ylabel = L"x_2", limits=
tricontourf!(ax1, x_samples[1, :], x_samples[2, :], y_samples[:, :], colormap=:bwr)
tricontourf!(ax2, x_samples[1, :], x_samples[2, :], y_NN[:, :], colormap=:bwr)
save("../figures/06p04TrainingMultivariateNeuralNetworks02.png", fig)
display(fig)
```



CairoMakie.Screen{IMAGE}

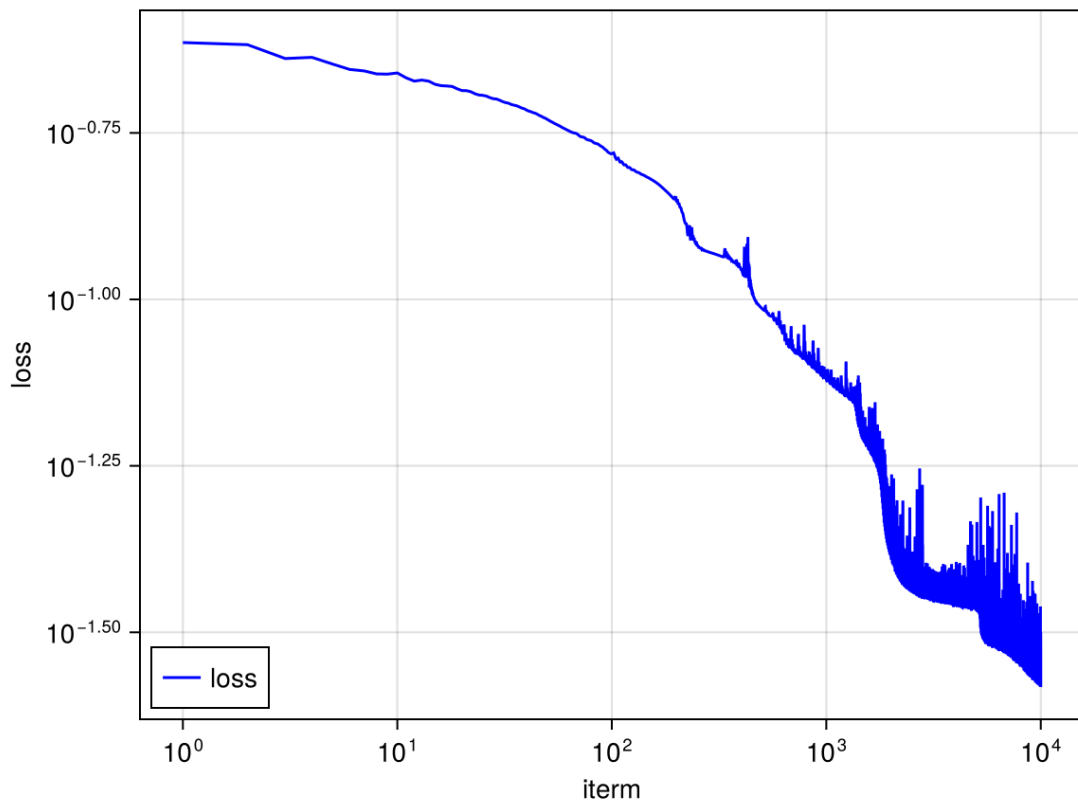
The left figure is the data and the right figure is the Neural Network prediction. As we can clearly see, we now have a much better prediction than using

$$f_{model}(x_1, x_2) = a_0 + a_1x_1 + a_2x_2 + a_3x_1x_2 + a_4x_1^2 + a_5x_2^2$$

Plot loss value vs iteration

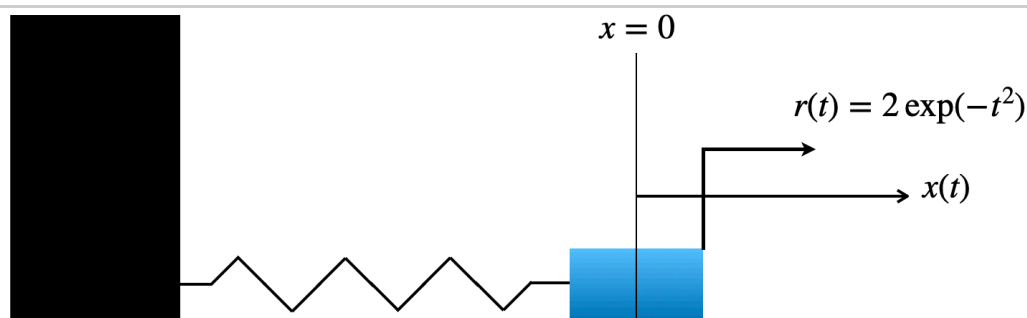
```
In [17]: fig = Figure()
ax = CairoMakie.Axis(fig[1, 1], xlabel = "item", ylabel = "loss",yscale=
lines!(ax,loss_history;color=:blue,label="loss")

axislegend(ax, position = :lb,orientation = :vertical)
fig
```



This is now better than the results you could obtain in Notebook 3.6

Many scientists/engineers would like to be able to do is to train a neural network on a particular dataset/parameter space and use it to predict at different parameters space. It would be great if we can make good predictions beyond the parameters the training data. However, this is often not the case. Neural Network usually do not do well if asked to predict for inputs that they have not seen. Let's have a look at the example below.



$$\frac{d^2x}{dt^2} + 4x = 2 \exp(-t^2)$$

$$x(0) = 0 \quad \frac{dx}{dt}(t = 1) = 0$$

You have a (blue) mass being pulled by a force $r(t) = 2e^{-t^2}$. The location of the mass, $x(t)$ is governed by the equation

$$\frac{d^2x}{dt^2} + 4x = 2e^{-t^2}$$

The physics is such that the velocity at at time $t = 1$, $dx/dt(t = 1) = 0$, i.e. the force somehow stops the mass at time $t=1$.

You have been given 2 sets of measurements, $x(t)$ for $t \in [0, 1]$ corresponding to the the initial value of

- $x(t = 0) = 0$ and
- $x(t = 0) = 1$

Let's now read and train from one data set and see if we can predict the data from the other data set.

```
In [18]: using CSV
using DataFrames

df = CSV.read("Data07.csv", DataFrame)
```

51×3 DataFrame

26 rows omitted

Row	xzero	tdata	xdata
	Float64	Float64	Float64
1	0.0	0.0	5.25802e-14
2	0.0	0.02	0.0267462
3	0.0	0.04	0.0542493
4	0.0	0.06	0.0824643
5	0.0	0.08	0.111344
6	0.0	0.1	0.140841
7	0.0	0.12	0.170905
8	0.0	0.14	0.201484
9	0.0	0.16	0.232525
10	0.0	0.18	0.263973
11	0.0	0.2	0.295774
12	0.0	0.22	0.32787
13	0.0	0.24	0.360204
⋮	⋮	⋮	⋮
40	0.0	0.78	1.10299
41	0.0	0.8	1.11794
42	0.0	0.82	1.13153
43	0.0	0.84	1.14372
44	0.0	0.86	1.15447
45	0.0	0.88	1.16376
46	0.0	0.9	1.17155
47	0.0	0.92	1.17783
48	0.0	0.94	1.18256
49	0.0	0.96	1.18573
50	0.0	0.98	1.18733
51	0.0	1.0	1.18733

```
In [19]: tdata=df[:, :tdata]
          xdata=df[:, :xdata]
          xzero=df[:, :xzero]
```

51-element Vector{Float64}:

```
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
⋮
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
0.0
```

This data set has inputs `xzero` and `tdata`. The output is `xdata`. Let's train a Neural Network to model this dataset

In order to use the Neural Network, we need to format the data so that the input data looks like

$$t_{input} = \begin{bmatrix} 0 & 0 & 0 & \dots & 0 \\ t_1 & t_2 & t_3 & \dots & t_N \end{bmatrix}$$

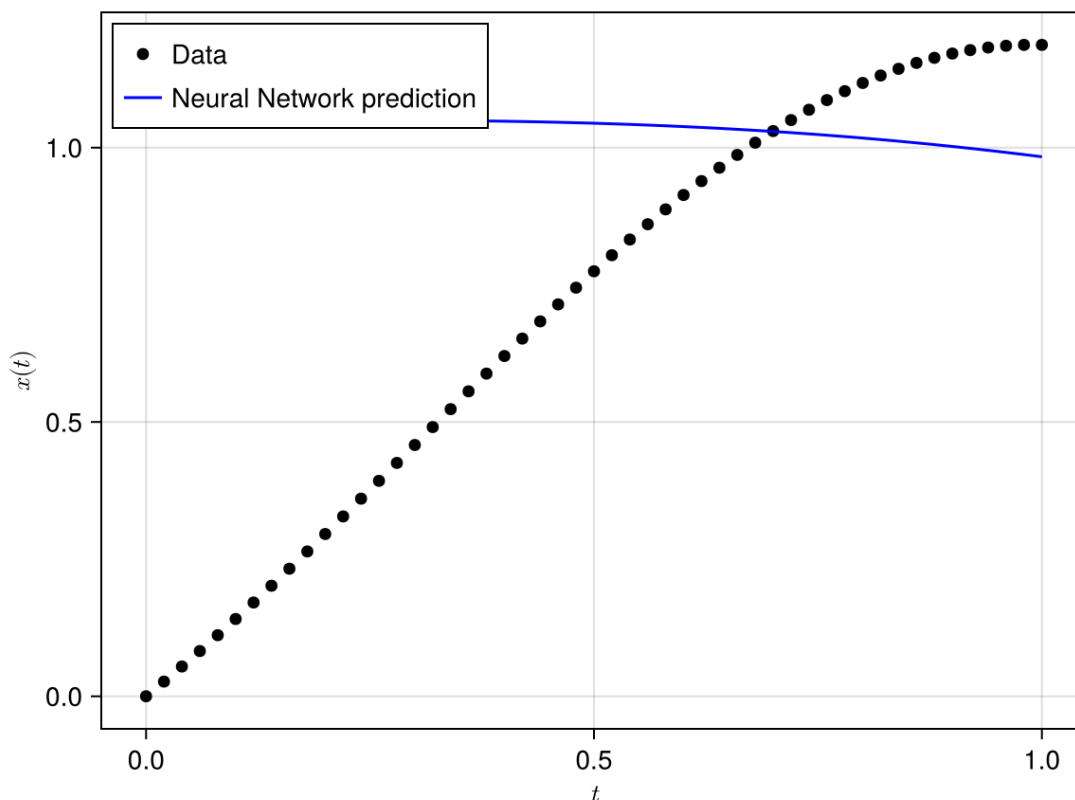
The zeros on the top row corresponds to the value of x at time equals to zero, i.e. $x(t = 0)$. The output data should look like

$$\begin{bmatrix} x_1 & x_2 & x_3 & \dots & x_N \end{bmatrix}$$

```
In [20]: t_input=[xzero';tdata']
model = Chain(
    Dense(2 => 5),
    Dense(5 => 5,tanh),
    Dense(5 => 1)
)
parameters, layer_states = Lux.setup(rng, model)
x_prediction, _ = model(t_input,parameters, layer_states)

fig = Figure()
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"t", ylabel = L"x(t)")
scatter!(ax,tdata,xdata;color=:black,label="Data")
lines!(ax,tdata,x_prediction[:];color=:blue,label="Neural Network predict")
axislegend(position=:lt)
fig
```

```
⌈ Warning: Mixed-Precision `matmul_cpu_fallback!` detected and Octavian.jl
cannot be used for this set of inputs (C [Matrix{Float64}]: A [Matrix{Floo
t32}] x B [Matrix{Float64}]). Falling back to generic implementation. This
may be slow.
└ @ LuxLib.Impl /Users/asho/.julia/packages/LuxLib/zPBrt/src/impl/matmul.j
l:194
```

The prediction is not good because we have not trained the Neural Network yet.

Set up the `loss_fn` function

```
In [21]: function loss_fn(p, ls)
           x_prediction, new_ls = model(t_input, p, ls)
           loss = 0.5 * mean((x_prediction .- xdata').^2)
           return loss
       end
```

`loss_fn` (generic function with 1 method)

Let's train the Neural Network on this data set

```
In [22]: callback = function (p, l)
           if length(loss_history) % 100 == 0
               println("Iteration: $(p.iter), Loss: $l")
           end
           push!(loss_history, l)
           return false
       end

adtype = Optimization.AutoZygote()
optf = Optimization.OptimizationFunction(loss_fn, adtype)
optprob = Optimization.OptimizationProblem(optf, ComponentArray(parameter

# Initialize a vector to store loss values
loss_history = Float64[]
neural_network = Optimization.solve(optprob, OptimizationOptimisers.Adam(
```

Iteration: 1, Loss: 0.13270670283732464
Iteration: 101, Loss: 0.0002195829965957153
Iteration: 201, Loss: 6.205694867705257e-5
Iteration: 301, Loss: 4.3463065855056435e-5
Iteration: 401, Loss: 3.0346280496018397e-5
Iteration: 501, Loss: 2.2325660776957894e-5
Iteration: 601, Loss: 1.7846180687330385e-5
Iteration: 701, Loss: 1.5381611022002838e-5
Iteration: 801, Loss: 1.3838949408021406e-5
Iteration: 901, Loss: 1.2610759059903855e-5
Iteration: 1001, Loss: 1.1441376908075665e-5
Iteration: 1101, Loss: 1.0260401099770373e-5
Iteration: 1201, Loss: 9.068262731192654e-6
Iteration: 1301, Loss: 7.881061916809846e-6
Iteration: 1401, Loss: 6.715669723757958e-6
Iteration: 1501, Loss: 5.593153193130644e-6
Iteration: 1601, Loss: 4.5430333027151875e-6
Iteration: 1701, Loss: 3.6014737398164448e-6
Iteration: 1801, Loss: 2.8030053691595755e-6
Iteration: 1901, Loss: 2.16948875063218e-6
Iteration: 2001, Loss: 1.701610533872104e-6
Iteration: 2101, Loss: 1.3783798501738651e-6
Iteration: 2201, Loss: 1.16522253098715e-6
Iteration: 2301, Loss: 1.0255596129366774e-6
Iteration: 2401, Loss: 9.295291768650639e-7
Iteration: 2501, Loss: 8.571105172910237e-7
Iteration: 2601, Loss: 7.969384052686383e-7
Iteration: 2701, Loss: 7.434679564810147e-7
Iteration: 2801, Loss: 8.828196306638166e-7
Iteration: 2901, Loss: 6.517239594087746e-7
Iteration: 3001, Loss: 6.112008058448997e-7
Iteration: 3101, Loss: 5.784459365978635e-7
Iteration: 3201, Loss: 5.441563905066271e-7
Iteration: 3301, Loss: 5.901950290732907e-6
Iteration: 3401, Loss: 4.996035709300185e-7
Iteration: 3501, Loss: 4.6983594171010284e-7
Iteration: 3601, Loss: 0.0001617025384142056
Iteration: 3701, Loss: 4.400323250804924e-7
Iteration: 3801, Loss: 4.235006688473857e-7
Iteration: 3901, Loss: 6.346527200970598e-7
Iteration: 4001, Loss: 4.240700171946946e-7
Iteration: 4101, Loss: 3.9607006509717485e-7
Iteration: 4201, Loss: 0.00013494435727003535
Iteration: 4301, Loss: 3.899851244786309e-7
Iteration: 4401, Loss: 3.804812863703619e-7
Iteration: 4501, Loss: 3.5582169024457094e-6
Iteration: 4601, Loss: 3.7843301128773926e-7
Iteration: 4701, Loss: 3.727872984200457e-7
Iteration: 4801, Loss: 5.783368980479374e-7
Iteration: 4901, Loss: 3.7451668025111284e-7
Iteration: 5000, Loss: 3.6984775905403415e-7

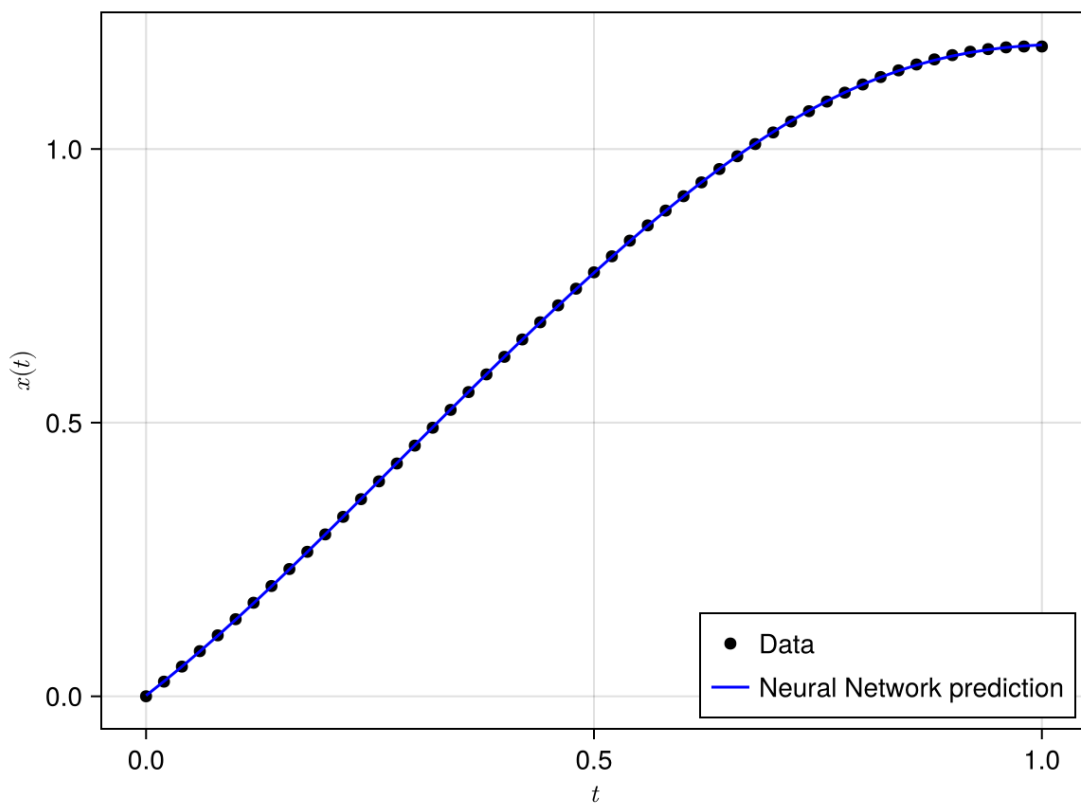
retcode: Default

```
u: ComponentVector{Float32}(layer_1 = (weight = Float32[0.9935731 1.094947
9; 0.19197696 -0.5294291; ... ; 0.20209746 0.7938749; -0.90211725 -0.1595270
6], bias = Float32[-0.76486814, -0.2778445, -0.4903074, 0.7549496, 0.51079
75]), layer_2 = (weight = Float32[-0.55811447 -0.92532825 ... -0.66088283 -
0.30874166; -2.091233 0.28885508 ... -0.35569742 1.1352026; ... ; 0.93508 0.83
279735 ... -0.6388073 0.10907046; -0.9134304 0.2309189 ... -1.1135938 0.066257
07], bias = Float32[0.46370885, -0.08690347, 0.43033978, 0.28756475, 0.362
73134]), layer_3 = (weight = Float32[0.5617595 -0.2708582 ... 0.51953876 -0.
77428937], bias = Float32[0.30278867]))
```

```
In [23]: x_prediction, new_ls= model(t_input,neural_network.u, layer_states)

([0.0013653268807202533 0.02732418552602813 ... 1.188407751229773 1.19025688
20454948], (layer_1 = NamedTuple(), layer_2 = NamedTuple(), layer_3 = Name
dTuple()))
```

```
In [24]: fig = Figure()
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"t", ylabel = L"x(t)")
scatter!(ax, tdata, xdata; color=:black, label="Data")
lines!(ax, tdata, x_prediction[:]; color=:blue, label="Neural Network predict")
axislegend(ax, position = :rb, orientation = :vertical)
fig
```



We have managed to fit the data pretty well!

Now let's read another data set. This data set is taken from the same physical system as the previous model. But only the value of x_{zero} has changed. Let's read "Data08.csv" and inspect its contents

```
In [25]: using CSV
using DataFrames

df = CSV.read("Data08.csv", DataFrame)
```

51×3 DataFrame

26 rows omitted

Row	xzero	tdata	xdata
	Float64	Float64	Float64
1	1.0	0.0	1.0
2	1.0	0.02	0.933765
3	1.0	0.04	0.866836
4	1.0	0.06	0.799318
5	1.0	0.08	0.731319
6	1.0	0.1	0.662945
7	1.0	0.12	0.594301
8	1.0	0.14	0.525496
9	1.0	0.16	0.456634
10	1.0	0.18	0.387822
11	1.0	0.2	0.319163
12	1.0	0.22	0.250762
13	1.0	0.24	0.182723
⋮	⋮	⋮	⋮
40	1.0	0.78	-1.19118
41	1.0	0.8	-1.21536
42	1.0	0.82	-1.23718
43	1.0	0.84	-1.25661
44	1.0	0.86	-1.27363
45	1.0	0.88	-1.28824
46	1.0	0.9	-1.30041
47	1.0	0.92	-1.31015
48	1.0	0.94	-1.31745
49	1.0	0.96	-1.32231
50	1.0	0.98	-1.32473
51	1.0	1.0	-1.32473

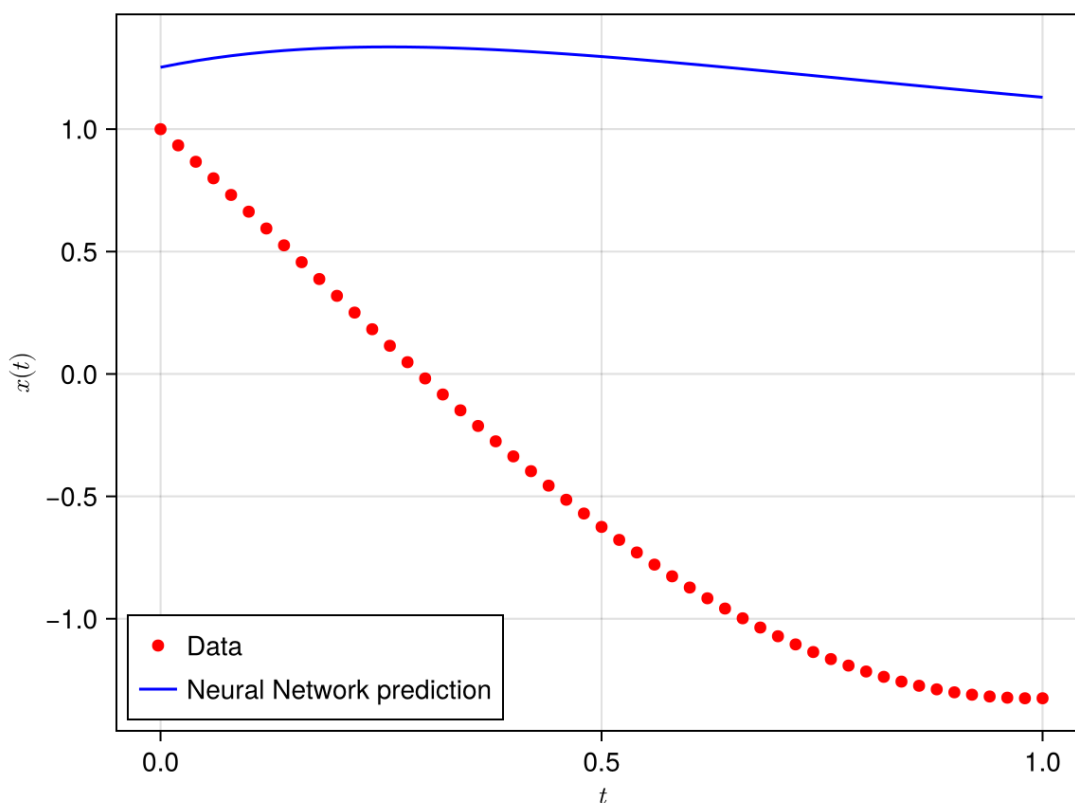
Observe that the value of `xzero` is now 1. Let's use the same Neural network to make the prediction and compare with the data from `Data08.csv`

In order to use the Neural Network we have just trained, we have to ensure that the input data looks like

$$t_{input} = \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ t_1 & t_2 & t_3 & \dots & t_N \end{bmatrix}$$

The value of 1 s on the first row is because the value of $x(t=0) = 1$.

```
In [26]: tdata=df[:, :tdata]
xdata=df[:, :xdata]
xzero=df[:, :xzero]
t_input=[xzero';tdata']
x_prediction, new_ls= model(t_input,neural_network.u, layer_states)
fig = Figure()
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"t", ylabel = L"x(t)")
scatter!(ax,tdata,xdata;color=:red,label="Data")
lines!(ax,tdata,x_prediction[:];color=:blue,label="Neural Network predict")
axislegend(ax, position = :lb,orientation = :vertical)
fig
```



As you can see, the Neural Network does not give a good prediction on data that it has not seen before. We have trained the data when `xzero` has a value of zero. However, when we use it to make a prediction for `xzero` equals to 1, it gives terrible predictions.

Now let's train our network on both $x(0) = 0$ and $x(0) = 1$ data sets and see if we can predict both data sets.

```
In [27]: using CSV
using DataFrames

df1 = CSV.read("Data07.csv", DataFrame)
tdata1=df1[:, :tdata]
xdata1=df1[:, :xdata]
xzero1=df1[:, :xzero]

df2 = CSV.read("Data08.csv", DataFrame)
tdata2=df2[:, :tdata]
xdata2=df2[:, :xdata]
xzero2=df2[:, :xzero]
```

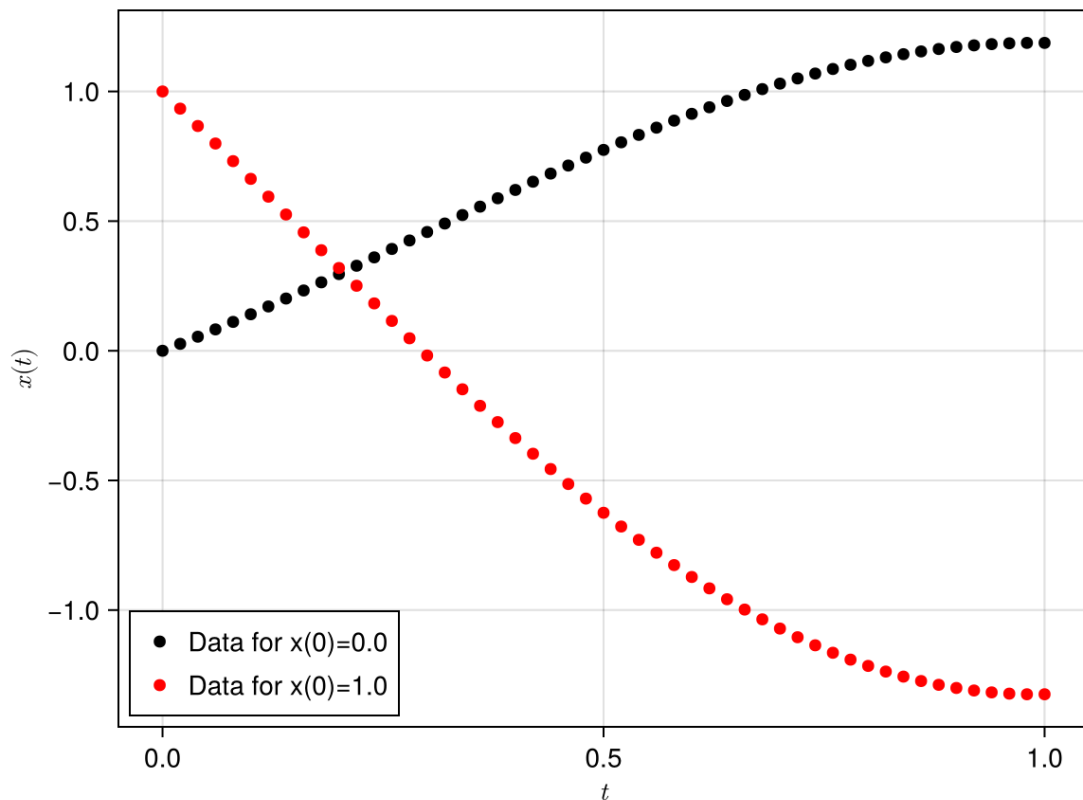
51-element Vector{Float64}:

```
1.0
1.0
1.0
1.0
1.0
1.0
1.0
1.0
1.0
1.0
⋮
1.0
1.0
1.0
1.0
1.0
1.0
1.0
1.0
1.0
```

Let's have a look at the two data sets

```
In [28]: fig = Figure()
ax1 = CairoMakie.Axis(fig[1, 1], xlabel = L"t", ylabel = L"x(t)")
scatter!(ax1,tdata1,xdata1;color=:black,label="Data for x(0)=0.0")
scatter!(ax1,tdata2,xdata2;color=:red,label="Data for x(0)=1.0")
axislegend(ax1, position = :lb,orientation = :vertical)

fig
```



Formatting the data for input into the Neural Network model for training. We need to put in both data from "Data07.csv" and "Data08.csv". The data has to be formatted as follows

$$t_{input} = \begin{bmatrix} 0 & 0 & 0 & \dots & 0 & 1 & 1 & 1 & \dots & 1 \\ t_1 & t_2 & t_3 & \dots & t_N & t_1 & t_2 & t_3 & \dots & t_N \end{bmatrix}$$

The output from the Neural Network would be

$$\begin{bmatrix} x_1^{(1)} & x_2^{(1)} & x_3^{(1)} & \dots & x_N^{(1)} & x_1^{(2)} & x_2^{(2)} & x_3^{(2)} & \dots & x_N^{(2)} \end{bmatrix}$$

```
In [29]: t_input=vcats(hcat(xzero1', xzero2'),hcat(tdata1', tdata2'))
          xdata_combined=hcat(xdata1', xdata2')
```

```
1×102 adjoint(::Vector{Float64}) with eltype Float64:
 5.25802e-14  0.0267462  0.0542493  ...  -1.32231  -1.32473  -1.32473
```

```
In [30]: model = Chain(
          Dense(2 => 5),
          Dense(5 => 5,tanh),
          Dense(5 => 1)
        )
          parameters, layer_states = Lux.setup(rng, model)
```

```
((layer_1 = (weight = Float32[1.183061 -0.61299354; -0.96052945 -0.1226557
05; ... ; 0.99058926 0.8809008; 0.55624646 -0.769506], bias = Float32[-0.573
87185, -0.07378067, 0.5426665, -0.29281884, -0.38988987]), layer_2 = (weig
ht = Float32[0.04001685 -0.022117373 ... -1.0208346 0.62015903; -0.08396796
-0.60337377 ... 0.9056151 -0.22010043; ... ; 0.9678946 -0.6085206 ... -0.6593714
4 -0.76693267; 0.23540057 1.0146302 ... 0.12261096 -1.1047238], bias = Float
32[0.43545607, -0.026276587, 0.07243845, 0.4435729, -0.39207837]), layer_3
= (weight = Float32[-0.11401739 0.75649184 ... 0.22037202 -0.45605305], bias
= Float32[-0.3306664])), (layer_1 = NamedTuple(), layer_2 = NamedTuple(),
layer_3 = NamedTuple()))
```

```
In [31]: function loss_fn(p, ls)
          x_prediction, new_ls= model(t_input, p, ls)
          loss = 0.5 * mean((x_prediction .- xdata_combined).^2)
          return loss
        end
```

loss_fn (generic function with 1 method)

```
In [32]: x_prediction, new_ls= model(t_input, parameters, layer_states)
          x_prediction.- xdata_combined
```

```
1×102 Matrix{Float64}:
 -0.253356 -0.239999 -0.227333 ... 2.22284 2.25493 2.28457 2.31171
```

```
In [33]: loss_fn(parameters, layer_states)
```

0.46653756860978174

```
In [34]: callback = function (p, l)
          if length(loss_history) % 100 == 0
            println("Iteration: $(p.iter), Loss: $l")
          end
          push!(loss_history, l)
          return false
        end

adtype = Optimization.AutoZygote()
optf = Optimization.OptimizationFunction(loss_fn, adtype)
optprob = Optimization.OptimizationProblem(optf, ComponentArray(parameter

# Initialize a vector to store loss values
loss_history = Float64[]
neural_network = Optimization.solve(optprob, OptimizationOptimisers.Adam(
```

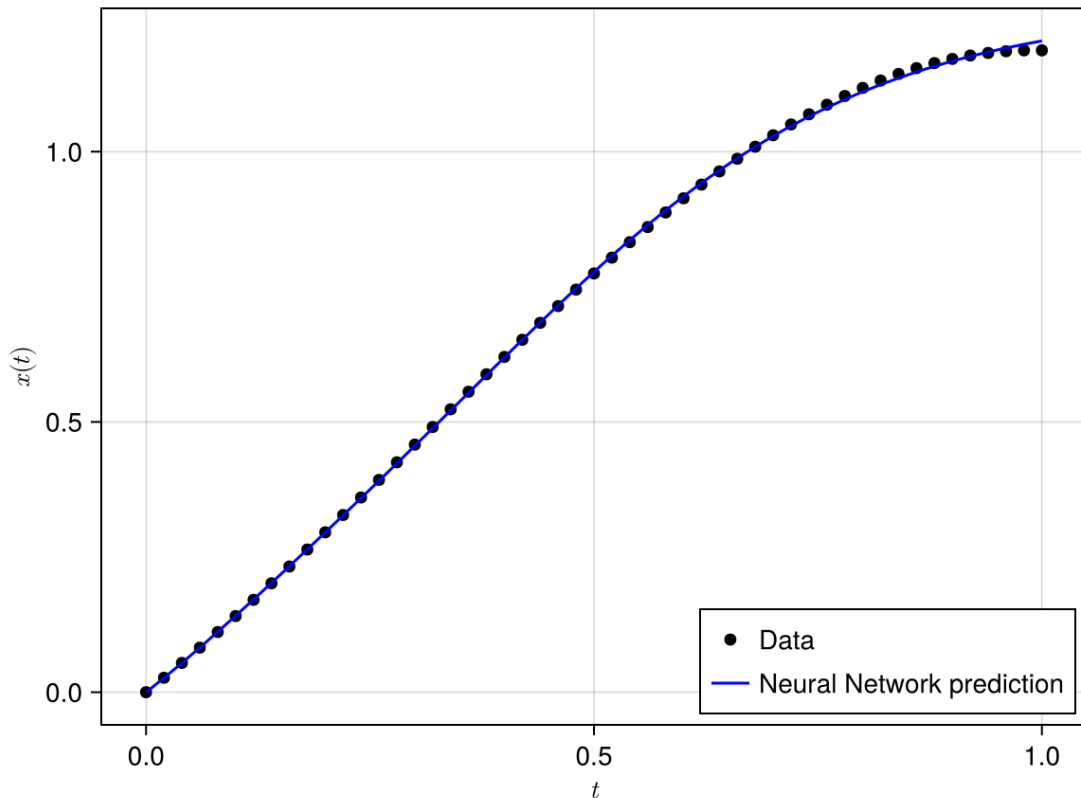

Iteration: 1, Loss: 0.46653756860978174
Iteration: 101, Loss: 0.00800901522463932
Iteration: 201, Loss: 0.0010672144155270185
Iteration: 301, Loss: 0.00045975393125279333
Iteration: 401, Loss: 0.00020113453387498738
Iteration: 501, Loss: 9.313640792046085e-5
Iteration: 601, Loss: 4.895311166143555e-5
Iteration: 701, Loss: 3.0220711025284967e-5
Iteration: 801, Loss: 2.1199768439976745e-5
Iteration: 901, Loss: 1.6285528565284326e-5
Iteration: 1001, Loss: 1.3532689770141038e-5
Iteration: 1101, Loss: 1.20519323878656e-5
Iteration: 1201, Loss: 1.1295614290370818e-5
Iteration: 1301, Loss: 1.0919359229066925e-5
Iteration: 1401, Loss: 1.0726672765811392e-5
Iteration: 1501, Loss: 1.0616059288834712e-5
Iteration: 1601, Loss: 1.0539385348088972e-5
Iteration: 1701, Loss: 1.0475579944928109e-5
Iteration: 1801, Loss: 1.0416065377740632e-5
Iteration: 1901, Loss: 1.0357590773720311e-5
Iteration: 2001, Loss: 1.029899017944531e-5
Iteration: 2101, Loss: 1.0239813949647852e-5
Iteration: 2201, Loss: 1.0179879932450473e-5
Iteration: 2301, Loss: 1.0119028859935964e-5
Iteration: 2401, Loss: 1.0057130968068391e-5
Iteration: 2501, Loss: 9.994056057159341e-6
Iteration: 2601, Loss: 9.929667463478604e-6
Iteration: 2701, Loss: 9.863825447400184e-6
Iteration: 2801, Loss: 9.796384875422042e-6
Iteration: 2901, Loss: 9.727206806561654e-6
Iteration: 3001, Loss: 9.656165608492637e-6
Iteration: 3101, Loss: 9.583122441974167e-6
Iteration: 3201, Loss: 9.507953165819513e-6
Iteration: 3301, Loss: 9.430546030472217e-6
Iteration: 3401, Loss: 9.350796239568542e-6
Iteration: 3501, Loss: 9.268614862903248e-6
Iteration: 3601, Loss: 9.183929607788233e-6
Iteration: 3701, Loss: 2.4589753272852607e-5
Iteration: 3801, Loss: 9.020787980304237e-6
Iteration: 3901, Loss: 8.929743274604507e-6
Iteration: 4001, Loss: 9.46188825576554e-6
Iteration: 4101, Loss: 8.767831154127962e-6
Iteration: 4201, Loss: 8.672337652338834e-6
Iteration: 4301, Loss: 8.714990878544145e-6
Iteration: 4401, Loss: 8.506139279830277e-6
Iteration: 4501, Loss: 2.5596301055624193e-5
Iteration: 4601, Loss: 8.34699114443e-6
Iteration: 4701, Loss: 8.246579406213199e-6
Iteration: 4801, Loss: 8.269001126339314e-6
Iteration: 4901, Loss: 8.097490324828356e-6
Iteration: 5000, Loss: 8.006808379881149e-6

retcode: Default

```
u: ComponentVector{Float32}(layer_1 = (weight = Float32[1.4419895 -0.30735
82; -1.2470273 0.043071177; ... ; 0.92265314 1.2984103; 0.6813292 -1.205730
4], bias = Float32[-0.52163833, -0.45106018, 0.68838406, -0.23965093, -0.2
9626292]), layer_2 = (weight = Float32[0.8690655 0.17651984 ... -1.1078001
1.5456083; -0.13954698 -0.93021685 ... 1.6369109 -0.51815945; ... ; 0.6159845
-0.66013706 ... -0.48256117 -1.2526798; 0.45916107 0.70591027 ... 0.77829236 -
1.6920831], bias = Float32[-0.34307396, 0.17928475, 0.254903, 0.86675596,
-0.5948779]), layer_3 = (weight = Float32[0.7740779 0.97001785 ... 0.4777485
4 -0.76818776], bias = Float32[-0.00067993026]))
```

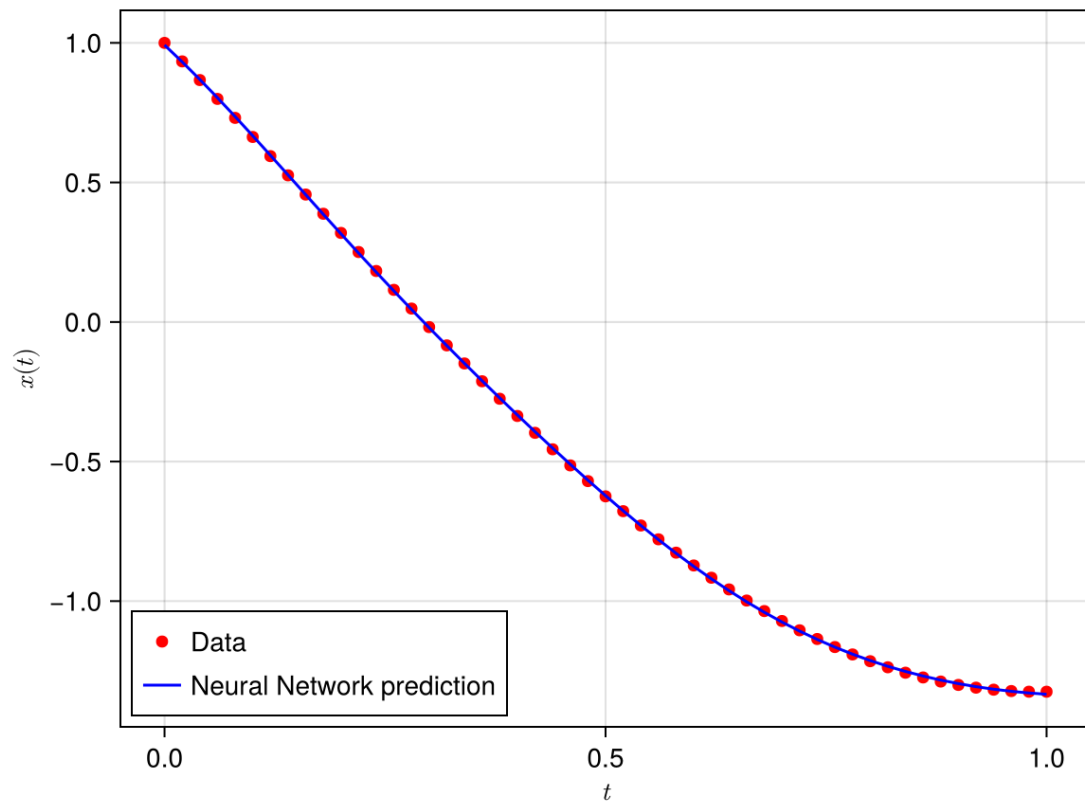
Now we predict the $x(0) = 0$ data.

```
In [35]: t_input1=[xzero1';tdata1']
x_prediction, new_ls= model(t_input1,neural_network.u, layer_states)
fig = Figure()
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"t", ylabel = L"x(t)")
scatter!(ax,tdata1,xdata1;color=:black,label="Data")
lines!(ax,tdata1,x_prediction[:];color=:blue,label="Neural Network predic
axislegend(ax, position = :rb,orientation = :vertical)
fig
```



Now we predict the $x(0) = 1$ data.

```
In [36]: t_input2=[xzero2';tdata2']
x_prediction, new_ls= model(t_input2,neural_network.u, layer_states)
fig = Figure()
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"t", ylabel = L"x(t)")
scatter!(ax,tdata2,xdata2;color=:red,label="Data")
lines!(ax,tdata2,x_prediction[:];color=:blue,label="Neural Network predic
axislegend(ax, position = :lb,orientation = :vertical)
fig
```



In [37]: